第6章课后习题 Q05～Q10

Q05 中央服务器为何会在授权前收到 RELEASE？如何解决？

这种情况可能出现在客户端请求被取消或消息延迟、乱序时。例如，进程 P 发送 REQUEST 后一直没有收到授权，于是超时放弃申请并发送 RELEASE；此时服务器可能还没处理到 P 的请求，或者 P 的请求仍在等待队列中。也可能是上一次访问的 RELEASE 延迟到达，而服务器已经收到了 P 下一次的 REQUEST。

我认为关键是不能把“收到 RELEASE”直接等同于“当前持锁者释放了锁”。可以给每次请求加上唯一的请求编号，服务器分别记录当前持锁者和等待队列：如果 RELEASE 对应的是等待中的请求，就取消该请求；只有它与当前持锁者的授权编号一致时才真正释放锁。对于已经发出但尚未被客户端确认的授权，还要处理取消与授权交叉到达的情况，避免错误地把锁再授予别人。

Q06 设计能容忍任一客户端崩溃的中央服务器互斥算法

假设服务器不会崩溃，客户端采用崩溃停止模型；同时假设系统是同步的，消息传输和正常处理时间有已知上界，使服务器能够可靠判断客户端是否崩溃。

服务器维护当前持锁者、请求队列和请求编号。客户端请求进入临界区时发送 REQUEST；服务器在锁空闲时发送 GRANT，否则加入队列。客户端使用完后发送 RELEASE，服务器核对持锁者和请求编号，再给下一个等待者授权。

为了处理客户端崩溃，服务器定期检测客户端状态。如果等待中的客户端崩溃，就删除它的请求；如果持锁客户端崩溃，就回收它的授权并处理下一个请求。对迟到的消息使用请求编号或授权世代号进行过滤。

安全性方面，服务器始终只记录一个有效持锁者，只有确认释放或确认持锁者已经崩溃才重新授权，因此不会让两个仍在运行的客户端同时合法持锁。活性方面，只要服务器持续运行、消息能在规定时间内送达，崩溃客户端就不会永久占用锁，其他正常客户端最终可以获得授权。

需要注意，若改为纯异步系统，单靠心跳超时无法确定客户端是真崩溃还是变慢。工程上通常进一步使用租约和单调递增的栅栏令牌，由资源端校验令牌，防止旧客户端恢复后继续写入；但栅栏保证的是资源操作安全，不能直接证明任意代码段绝不重叠执行。

Q07 同步系统中如何在最多 f 个进程崩溃时达成共识？最少需要多少轮？

可以采用课件介绍的同步洪泛算法。假设有 N 个进程，N > f，最多 f 个进程会崩溃；消息在每一同步轮结束前送达尚未崩溃的接收者。

1. 每个进程先把自己的初始提议加入本地集合 V。
2. 每一轮向其他进程发送当前的 V，并把这一轮收到的提议合并到自己的 V 中。
3. 连续进行 f+1 轮后，每个正确进程都从自己的 V 中选取最小值作为决定值。

终止性：同步轮有明确结束时间，所以正确进程完成 f+1 轮后都能决定。

协定性：如果某个提议只传给了部分进程，下一轮知道该提议的进程会继续传播。要让正确进程到第 f+1 轮结束后仍掌握不同的提议集合，最坏情况下需要连续出现超过 f 次崩溃来阻断传播，这与最多 f 次崩溃矛盾。因此正确进程最终拥有相同的集合，并决定相同的最小值。

有效性：决定值始终来自某个进程最初提出的值，不会凭空产生新值；即使最初提出该值的进程后来崩溃，也不影响这一性质。

该算法在最坏情况下需要 f+1 轮。这里的轮数结论依赖同步系统和崩溃停止模型，不能直接用于纯异步网络。

Q08 N=5 时读写法定数如何选择?

设读操作需要 R 个副本响应，写操作需要 W 个副本响应，总副本数 N=5。为使任意读集合与写集合相交，需要 R+W>5；为使任意两个写集合相交，需要 2W>5，因此 W 至少为 3。

读法定数 R	满足条件的最小写法定数 W	R+W	2W	是否满足两种交集
1	5	6	10	是
2	4	6	8	是
3	3	6	6	是
4	3	7	6	是
5	3	8	6	是

例如 R=3、W=3 比较均衡：任意两个大小为 3 的写集合至少有一个共同节点，任意大小均为 3 的读写集合也至少有一个共同节点。若选择 R=1、W=5，读取更容易，但每次写入必须联系全部副本，可用性较差。

我理解交集只是实现一致性的必要结构条件，并不表示数据已经自动一致。实际系统还要处理版本号、并发写入、旧数据识别、写入完成条件及故障恢复，否则即使集合相交，也可能读到旧版本。

Q09 五节点 Raft 集群分区并恢复后的变化

设集群为 S1～S5，多数派门槛为 3 票，初始任期 term=7，S1 是 Leader。随后网络分为 {S1,S2} 和 {S3,S4,S5} 两部分。假设多数派一侧有满足 Raft 日志新旧条件的候选者，且成功完成选举。

阶段	{S1,S2} 一侧	{S3,S4,S5} 一侧
分区前	S1 为 term 7 的 Leader	其他节点跟随 S1
刚发生分区	S1 可能暂时仍认为自己是 Leader，但最多只能得到 2 个节点响应	收不到原 Leader 心跳，节点可能超时发起选举
新任期选举	只有 2 票，无法形成多数派	例如 S3 在 term 8 获得 S3、S4、S5 共 3 票，成为新 Leader
分区期间写入	S1 无法把新日志复制到至少 3 个节点，不能提交新的日志项	新 Leader 可按 Raft 的日志复制和提交规则处理写入
网络恢复	S1 收到更高任期 term 8 的有效消息，更新任期并退回 Follower	S3 继续在 term 8 协调集群

这个过程说明，Raft 依靠“任期 + 多数派投票”保证同一任期最多选出一位 Leader。因为任意两个 3 票多数派必然相交，而一个节点在同一任期不能投两票，所以不会在同一任期产生两个合法的 Leader。

但也要区分“自认为是 Leader”和“有能力提交新日志”：旧主 S1 在分区期间可能仍保留 Leader 状态，却没有多数派支持，不能完成新写入的提交。恢复后它会根据更高任期退位。仅靠选举机制本身还不够，写入安全也需要 Raft 的日志检查与提交规则。

Q10 FLP 不可能性为何不妨碍工程中实现共识？（150～250 字）

FLP定理讨论的是纯异步、确定性协议，并允许至少一个进程崩溃的情况。在这些前提下，协议无法保证每一种可能的执行都最终决定，但不代表所有执行都会失败。工程系统通常通过增加条件获得进展，例如采用部分同步模型，假设网络最终趋于稳定，消息延迟最终受到约束；同时用多数派避免冲突。Raft利用随机选举超时降低分票概率，在多数节点可通信且网络逐渐稳定时可以完成选主和日志提交。但随机超时不等于在纯异步环境下证明必然终止。因此，FLP说明的是理论保证的边界，没有否定工程中有条件地实现共识。